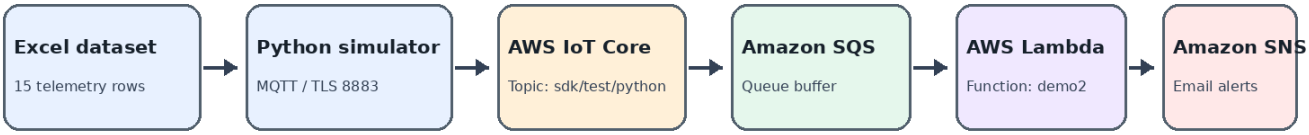
SMART GRID IoT

AWS Workshop Handbook

A step-by-step guide to streaming simulated grid telemetry and sending warning / critical email alerts

END-TO-END SMART GRID ALERT PIPELINE



Optional parallel check: AWS IoT Core MQTT Test Client subscribes to sdk/test/python.

Rule SQL: SELECT * FROM 'sdk/test/python' | SQS event source mapping invokes Lambda.

Pipeline: Excel dataset → Python simulator → AWS IoT Core → IoT Rule → SQS → Lambda (demo2) → SNS → Email

Prepared from the supplied Smart Grid IoT guide, policy, dataset and Lambda code, plus the stated workshop flow.

Console visuals throughout this handbook are illustrative schematics, not screenshots captured from a live AWS account. AWS console labels may change.

How to use this handbook

This handbook follows the workflow in order. If the AWS resources have already been created, do not recreate them: start at the verification checkpoints and only change what is missing. Use one AWS region consistently; the supplied guide uses Asia Pacific (Mumbai), ap-south-1.

Workshop outcome

A local Python program reads 15 smart-grid rows from Excel and publishes each as JSON to the MQTT topic `sdk/test/python`. An AWS IoT rule forwards each matching message to an SQS queue. SQS invokes Lambda function `demo2`. Lambda classifies each record as Normal, Warning, or Critical and publishes Warning/Critical notifications to an SNS topic with an email subscription.

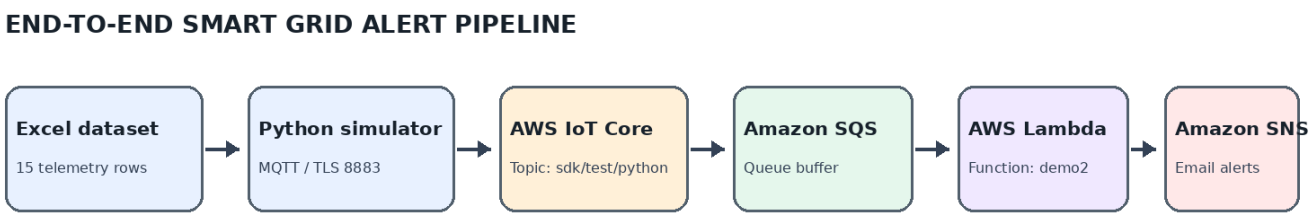
Important source alignment notes

• The uploaded guide documents the simulator → IoT Core portion and the topic/client/certificate configuration. The uploaded Lambda code expects an SQS event with `Records[].body` and publishes to SNS. The workshop flow connects those pieces using an IoT rule and SQS event source mapping.
• The shared ChatGPT links could be identified by title, but their full conversation bodies were not readable in this environment. This handbook therefore does not invent unverified details from those pages; it is grounded in the available uploaded guide, policy, dataset, Lambda code, and prior workflow context.
• Console images are labelled schematics to show where fields appear. They are not actual captures of your AWS account.

Before you begin

Check	Requirement
AWS account	Permission to manage IoT Core, IAM roles, SQS, Lambda, SNS and CloudWatch.
Region	Use ap-south-1 consistently unless your existing resources are in another region.
Files	smart_grid_iot_simulator.py, smart_grid_dataset_15.xlsx, Root CA PEM, device certificate CRT, private KEY.
Email	Access to the inbox used for the SNS email subscription.
Security	Never upload or share the private key; avoid placing account IDs or ARNs in public materials.

1. Understand the end-to-end architecture



Optional parallel check: AWS IoT Core MQTT Test Client subscribes to sdk/test/python.

Rule SQL: SELECT * FROM 'sdk/test/python' | SQS event source mapping invokes Lambda.

What each service does

Component	Purpose in this workshop
Excel dataset	Contains 15 sample records with grid measurements and status.
Python simulator	Reads Excel rows and publishes JSON messages through MQTT over TLS.
AWS IoT Core	Secure MQTT broker; receives telemetry on sdk/test/python.
IoT Rule	Matches incoming MQTT messages and forwards them to SQS.
Amazon SQS	Buffers messages until Lambda processes them.
AWS Lambda: demo2	Reads each SQS body, checks temperature/current/status, and publishes alerts.
Amazon SNS	Delivers warning/critical messages to confirmed email subscriptions.
CloudWatch Logs	Shows Lambda execution, decisions and errors.

Data path and verification path

Verify the pipeline in sequence, rather than testing only the final email: (1) simulator terminal, (2) IoT MQTT Test Client, (3) SQS message metrics, (4) Lambda CloudWatch logs, (5) SNS email. The first checkpoint that fails usually identifies the broken connection.

2. Prepare your computer and project folder

Required software and libraries

Item	Why it is needed	Install / check
Python 3.8+	Runs the simulator script.	python3 --version
AWS IoT Device SDK for Python v2	Secure MQTT connection and certificate handling.	pip install awsiotsdk
pandas	Loads Excel rows into a DataFrame.	pip install pandas
openpyxl	Reads .xlsx files through pandas.	pip install openpyxl
boto3	Used by Lambda code to call SNS; available in the Lambda Python runtime in standard configurations.	No local install needed for Lambda; local tests may use pip install boto3.
AWS CLI (optional)	Useful for checking identity and automating AWS tasks; the pipeline can be configured through the console.	aws --version; aws sts get-caller-identity

Recommended project folder

```
smart-grid-workshop/
■■■ smart_grid_iot_simulator.py
■■■ smart_grid_dataset_15.xlsx
■■■ AmazonRootCA1.pem
■■■ device-certificate.pem.crt
■■■ device-private.pem.key
■■■ venv/   (created locally; do not share)
```

Create a virtual environment

Open a terminal in the folder containing the simulator and dataset. On Linux/macOS:

```
python3 -m venv venv
source venv/bin/activate
python -m pip install --upgrade pip
python -m pip install awsiotsdk pandas openpyxl
python -c "from awsiot import mqtt_connection_builder; import pandas, openpyxl; print('Dependencies OK')"
```

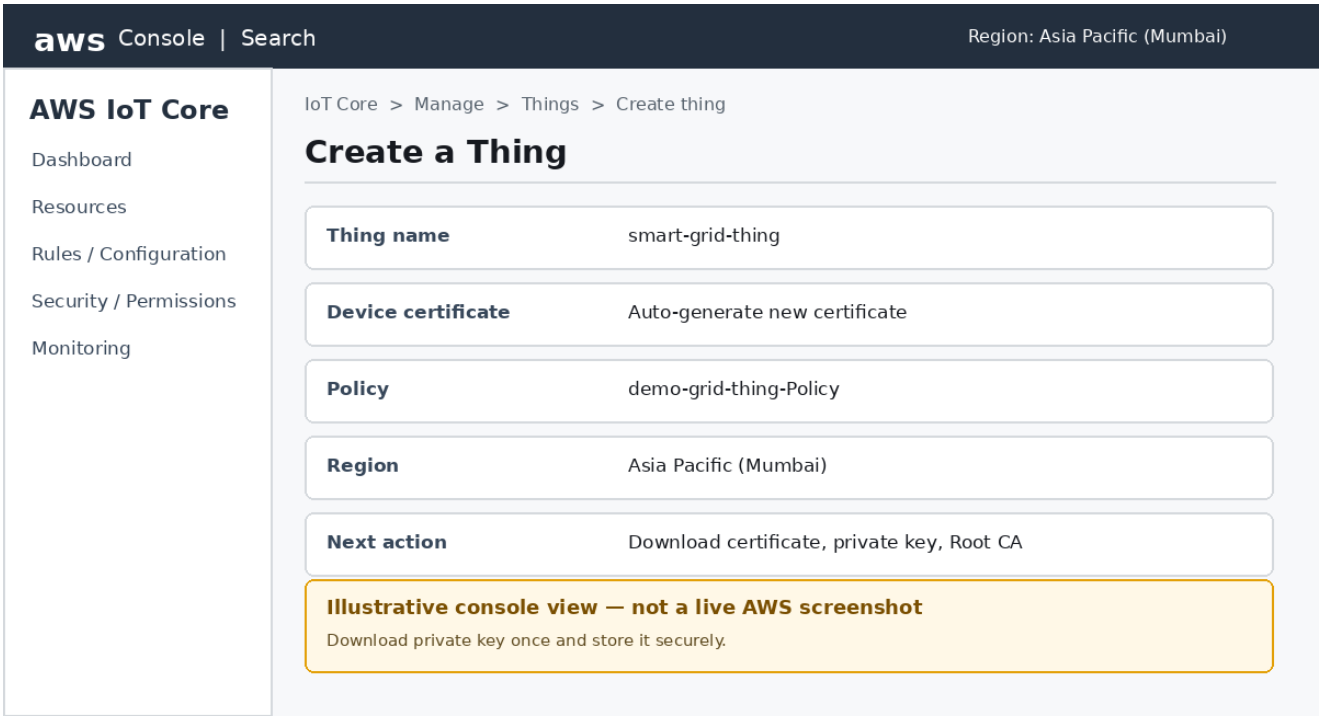
On Windows PowerShell:

```
py -m venv venv
.\venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install awsiotsdk pandas openpyxl
python -c "from awsiot import mqtt_connection_builder; import pandas, openpyxl; print('Dependencies OK')"
```

Run the script using the same environment where packages were installed. On Debian/Kali systems, avoid system-wide pip installs; use the virtual environment to avoid externally-managed-environment errors.

3. Configure AWS IoT Core: Thing, certificate and policy

If your Thing and certificate already exist, verify their status rather than creating duplicates.

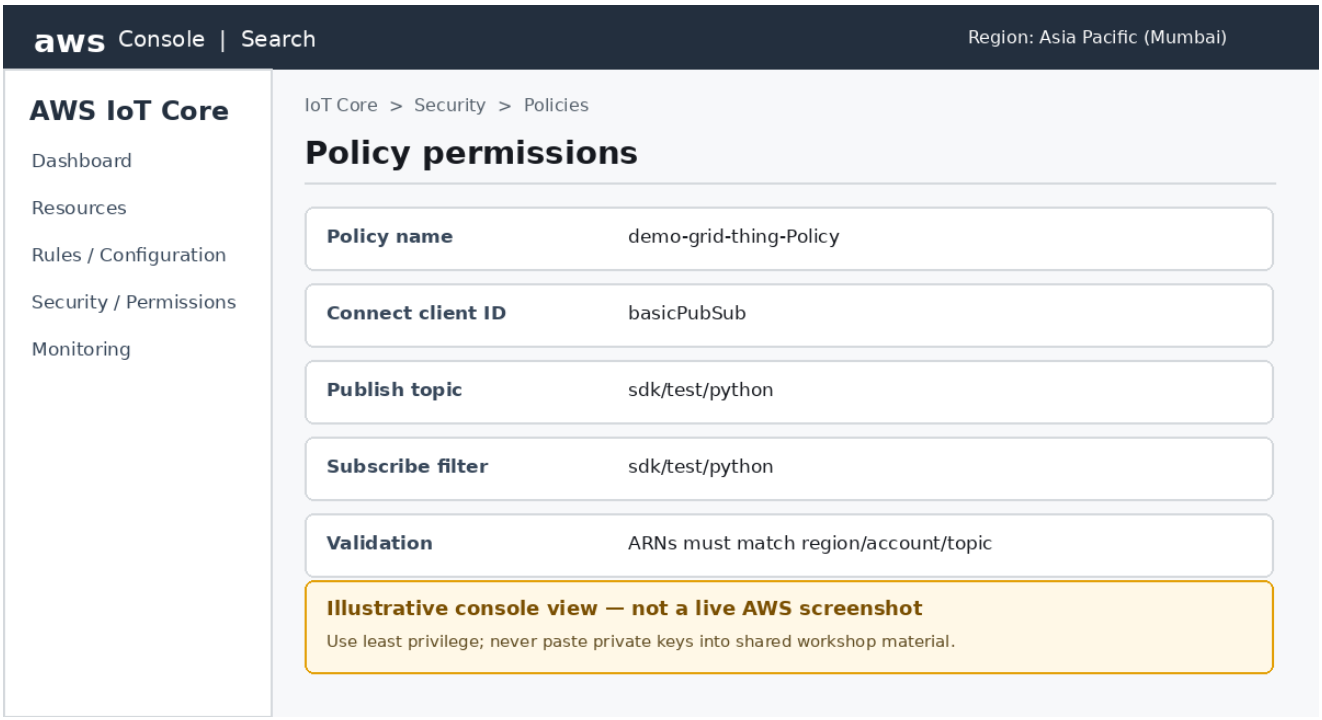


Visual guide: Create Thing wizard: name, certificate generation and policy association. (schematic, not a live AWS screenshot)

Steps

1. Sign in to the AWS Console and set the region to Asia Pacific (Mumbai) / ap-south-1 if that is where your resources live.
2. Open AWS IoT Core → Manage → Things (console navigation wording may vary). Choose Create things → Create single thing.
3. Name it `smart-grid-thing` (or retain the existing Thing name used by your team).
4. Generate a new device certificate if needed. Download the device certificate, private key and Amazon Root CA certificate. Store the private key securely; AWS generally does not let you download it again later.
5. Attach the intended IoT policy to the certificate. If the console workflow offers policy selection during Thing creation, select the correct policy there; otherwise attach it from the certificate's security details.

Review the IoT policy



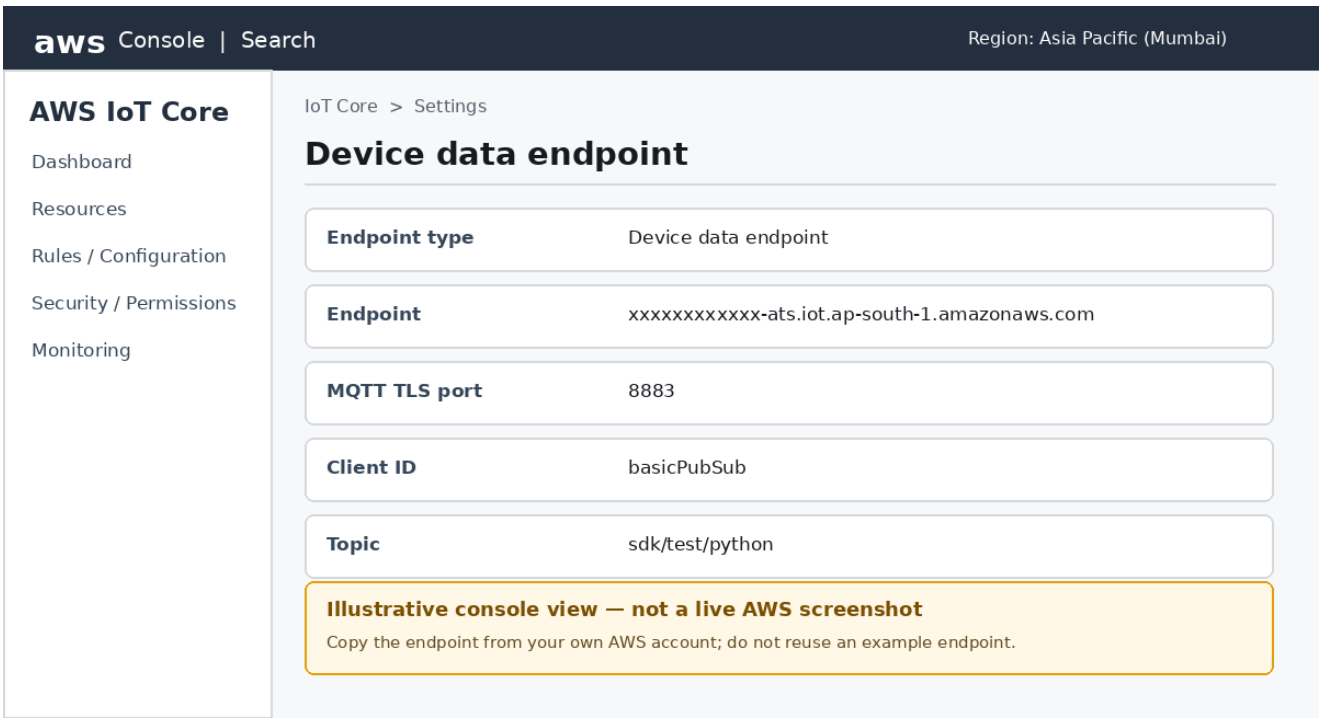
Visual guide: Policy page: confirm Connect client ID and Publish topic permissions. (schematic, not a live AWS screenshot)

The supplied policy allows publishing to `sdk/test/python` and includes `basicPubSub` as an allowed client ID. Its resource ARNs include an account-specific ID in the uploaded file; when creating a policy in another account, replace the account ID with your own account ID and verify the region and resource names. Do not copy account-specific ARNs blindly.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["iot:Connect"],
      "Resource": ["arn:aws:iot:ap-south-1:ACCOUNT_ID:client/basicPubSub"]
    },
    {
      "Effect": "Allow",
      "Action": ["iot:Publish"],
      "Resource": ["arn:aws:iot:ap-south-1:ACCOUNT_ID:topic/sdk/test/python"]
    }
  ]
}
```

This is a minimal illustrative policy for the publisher only. If your test client also needs subscribe permissions, add the required iot:Subscribe and iot:Receive permissions for the appropriate topic filters. Preserve the existing policy if it is already tested and correctly scoped.

4. Copy the device endpoint and verify certificates



Visual guide: Settings page: use the endpoint shown in your own AWS IoT Core account. (schematic, not a live AWS screenshot)

1. Open AWS IoT Core → Settings and copy the Device data endpoint. It usually ends in `-ats.iot.ap-south-1.amazonaws.com`.
2. Paste it into the simulator's `AWS_ENDPOINT` setting. Do not use a sample endpoint from a guide.
3. Open IoT Core → Security → Certificates. Confirm the device certificate is Active and that the intended policy is attached.
4. Check that the paths in the Python script exactly match the downloaded filenames for the Root CA, device certificate and private key.
5. Do not put the private key into a screenshot, slide, public repository or shared handbook.

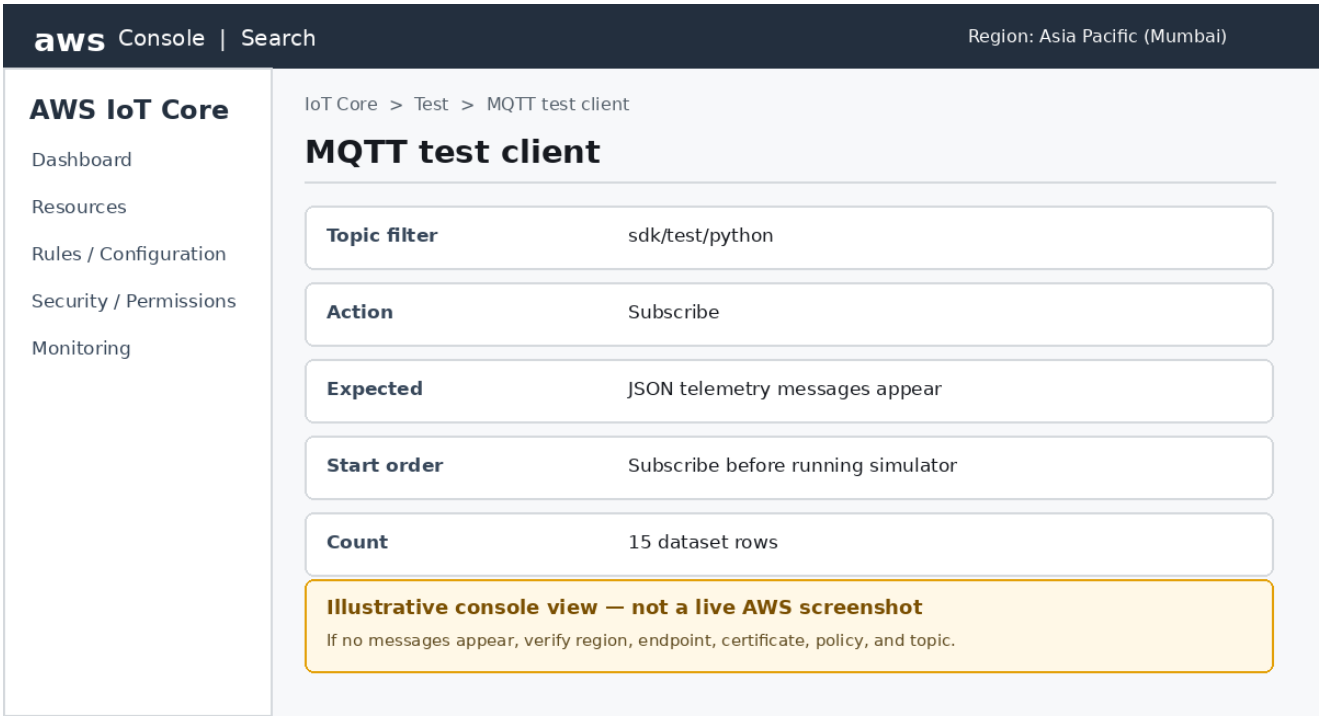
Simulator configuration values to check

Setting	Expected value / rule
AWS_ENDPOINT	Copied from your own IoT Core Settings page
CLIENT_ID	basicPubSub (must be allowed by iot:Connect)
MQTT_TOPIC	sdk/test/python (must be allowed by iot:Publish)
PATH_TO_ROOT_CA	Path to AmazonRootCA1.pem
PATH_TO_CERT	Path to device certificate .crt
PATH_TO_KEY	Path to device private .key
DATASET_PATH	Path to smart_grid_dataset_15.xlsx
STREAM_DELAY_SECONDS	1.0 seconds per message in the supplied guide

The uploaded guide uses client ID `basicPubSub`, topic `sdk/test/python`, and a one-second delay. If you change the client ID or topic, update the policy to match exactly.

5. Run the simulator and confirm MQTT messages

Start the MQTT Test Client first



Visual guide: MQTT Test Client: subscribe to sdk/test/python before starting the simulator. (schematic, not a live AWS screenshot)

1. Open AWS IoT Core → Test → MQTT test client.
2. Choose Subscribe to a topic.
3. Enter `sdk/test/python` and click Subscribe.
4. In your local terminal, run the simulator from the project folder:

```
# Linux/macOS
source venv/bin/activate
python smart_grid_iot_simulator.py

# Windows PowerShell (after activating venv)
python .\smart_grid_iot_simulator.py
```

Expected result

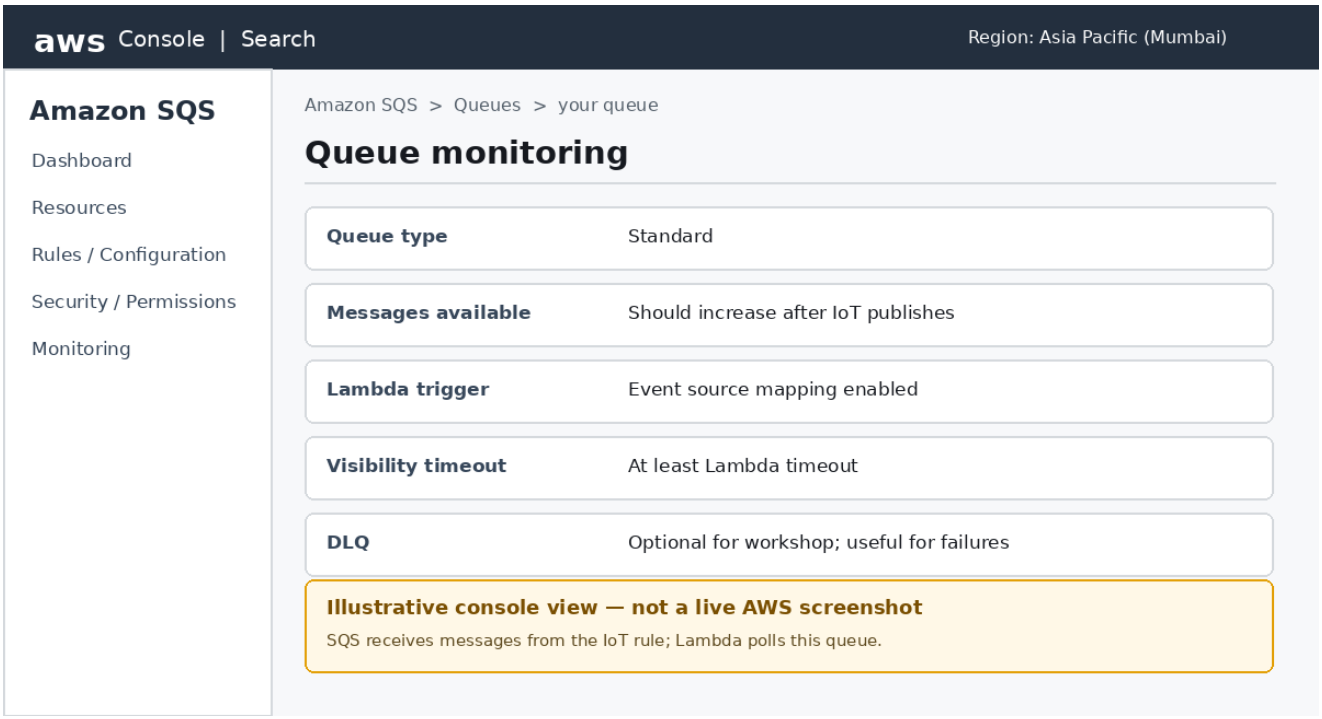
The terminal should report that 15 records were loaded, a secure connection was established, and each row was published to `sdk/test/python`. The MQTT Test Client should show incoming JSON messages. The sample workbook has 15 records: 8 Normal, 4 Warning and 3 Critical statuses.

Payload fields

Field	Meaning
Grid_ID	Grid / substation identifier
Timestamp	Reading time
Voltage_kV, Current_A	Voltage and load current
Frequency_Hz	Grid frequency
Active_Power_MW, Reactive_Power_MVAR	Power measurements
Power_Factor	Power factor

Field	Meaning
Transformer_Temp_C	Transformer temperature
Load_Percent, Energy_MWh	Load and energy
Breaker_Status	Breaker state
Grid_Status	Normal / Warning / Critical

6. Create the SQS queue



Visual guide: SQS queue page: confirm the queue exists and monitor messages. (schematic, not a live AWS screenshot)

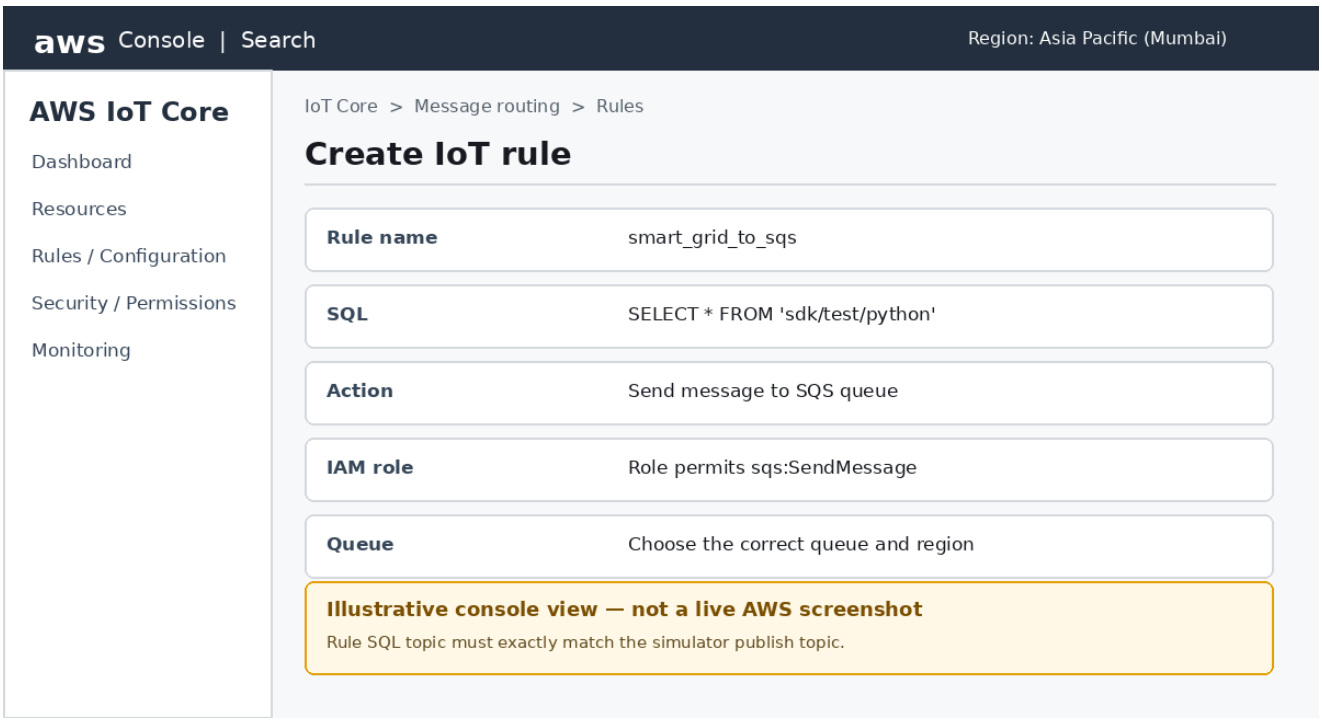
1. Open Amazon SQS → Queues → Create queue if a queue has not already been created. A Standard queue is sufficient for this demonstration.
2. Give it a descriptive name, for example `smart-grid-telemetry-queue`. If the workflow already has a queue connected to Lambda, keep using that queue instead of creating another one.
3. Keep the queue in the same AWS region as the IoT rule and Lambda function.
4. Review visibility timeout. It should be longer than the Lambda function timeout to reduce the chance of a message becoming visible again while Lambda is still processing it.
5. Save the queue and copy its ARN; the IoT rule action and Lambda trigger must refer to the intended queue.

Permissions to verify

The IoT rule needs permission to send messages to the queue. In the IoT rule configuration, select or create an IAM role that allows `sqs:SendMessage` to this queue. If the queue has a restrictive resource policy, it must also permit the rule's role to send messages. Lambda needs permission to poll the queue through the event source mapping; the Lambda execution role normally needs the SQS receive/delete/attributes permissions required for that mapping.

Do not confuse these permissions: the IoT rule writes to SQS; Lambda reads from SQS. These are separate actions and may use separate IAM roles.

7. Create the AWS IoT rule to forward telemetry to SQS



Visual guide: IoT Rule editor: SQL statement and SQS action. (schematic, not a live AWS screenshot)

1. Open AWS IoT Core → Message routing → Rules (older console layouts may show Rules under Act).
2. Choose Create rule. Use a name such as `smart_grid_to_sqs`.
3. Enter the SQL statement below. The topic must match the simulator exactly:

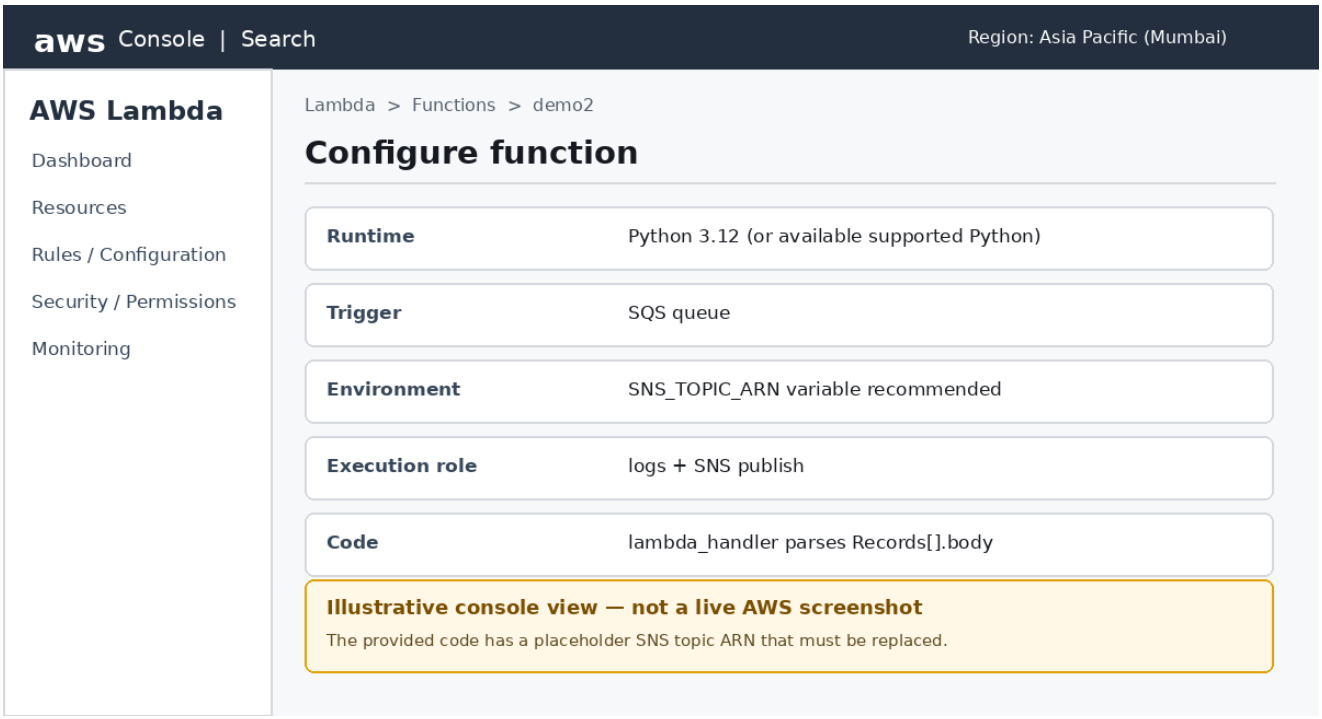
```
SELECT * FROM 'sdk/test/python'
```

4. Add an action to Send a message to an Amazon SQS queue. Select the intended queue ARN.
5. Choose the IAM role that grants `sqs:SendMessage` to that queue. Create a dedicated role if needed and follow the console's role-creation flow.
6. Save / create the rule and ensure it is enabled.
7. Publish a new test message by rerunning the simulator (or use the MQTT Test Client publish panel with a valid JSON payload).
8. Open SQS and check message metrics / receive messages. If the queue remains empty, inspect the IoT rule's status, SQL topic filter, action role and queue policy.

Message format warning

The Lambda code expects the SQS body to contain a JSON object with fields such as `Grid_ID`, `Transformer_Temp_C`, `Current_A` and `Grid_Status`. The IoT rule's SQS action should forward the telemetry message as the message body, not wrap it in a different custom JSON structure unless you also update the Lambda code.

8. Configure Lambda function demo2



Visual guide: Lambda function page: code, SQS trigger and execution role. (schematic, not a live AWS screenshot)

1. Open AWS Lambda → Functions → demo2. Use the existing function if it is already connected to SQS.
2. Set a supported Python runtime available in your account. The provided handler uses standard Python plus boto3.
3. Under Code, paste or upload the supplied Lambda code and save / deploy it.
4. Replace `YOUR_SNS_TOPIC_ARN_HERE` with the full ARN of the SNS topic you intend to publish to. Safer alternative: use an environment variable named `SNS_TOPIC_ARN` and read it in the code.
5. Under Configuration → Permissions, inspect the execution role. It needs CloudWatch Logs permissions and `sns:Publish` on the target SNS topic.
6. Under Configuration → Triggers, add or verify the SQS queue. The event source mapping must be enabled. Wait for the trigger to become active before testing.

Lambda logic from the supplied code

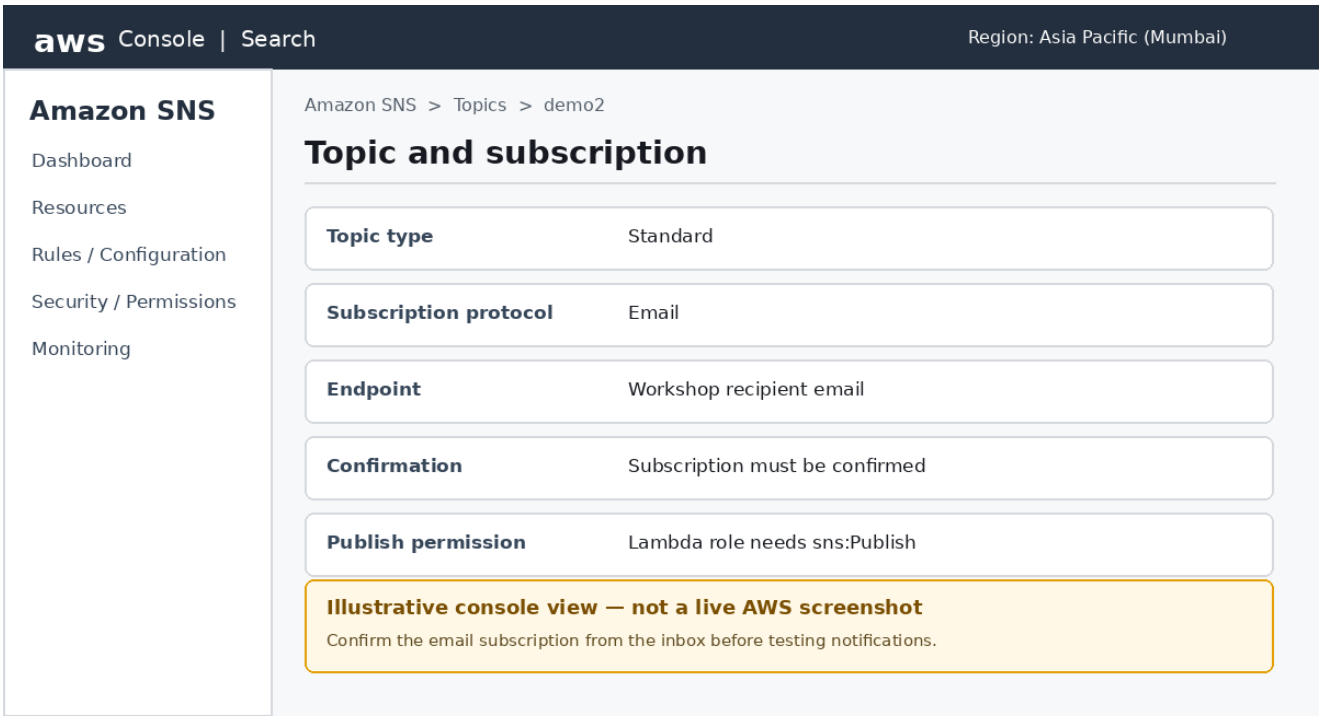
Condition (checked in this order)	Outcome
Grid_Status == Critical OR temp ≥ 80°C OR current ≥ 600 A	Publish critical fault alert to SNS
Else if Grid_Status == Warning OR temp ≥ 65°C OR current ≥ 500 A	Publish warning alert to SNS
Otherwise	Log that the grid is operating safely; no SNS message

Because the critical condition is evaluated first, a record with high critical thresholds is treated as Critical even if it also meets warning thresholds. The code's fallback values are 0 for temperature/current and Normal for status when fields are missing; a missing field may therefore hide a problem, so inspect the payload schema if alerts seem wrong.

Code safety and maintainability

Avoid hardcoding resource identifiers in code where possible. An environment variable makes it easier to switch between workshop accounts without editing source code. Keep the IAM policy limited to the single SNS topic rather than granting sns:Publish to every topic.

9. Configure SNS topic and email subscription



Visual guide: SNS topic and subscription: confirm the recipient email. (schematic, not a live AWS screenshot)

1. Open Amazon SNS → Topics and select the existing topic named `demo2`, if that is the topic used by the Lambda code. If no topic exists, create a Standard topic and use its ARN in Lambda.
2. Under Subscriptions, create a subscription with protocol Email and enter the recipient email address.
3. Open the inbox and click Confirm subscription. An unconfirmed email subscription will not receive notifications.
4. Copy the topic ARN and ensure it exactly matches the ARN configured in Lambda.
5. Confirm Lambda's execution role can publish to that topic: `sns:Publish` on the topic ARN.
6. Test the topic itself with a harmless sample message if you need to verify email delivery independently from the pipeline.

What the email should contain

Critical alert subject: `EMERGENCY ALARM: GRID_ID Critical Failure`. The message includes grid location, status, transformer temperature, load current and the recommended action. Warning alert subject begins with `WARNING` and asks for technician inspection. Emoji rendering can vary by email client.

Operational note

This is a workshop simulation. The Lambda message says “Immediate shutdown and maintenance required” for a critical event, but the pipeline itself does not shut down physical equipment. Do not treat this demonstration as a production protection or control system.

10. Test the complete pipeline end to end

Use the following sequence after all services are configured. Testing one hop at a time makes faults much easier to isolate.

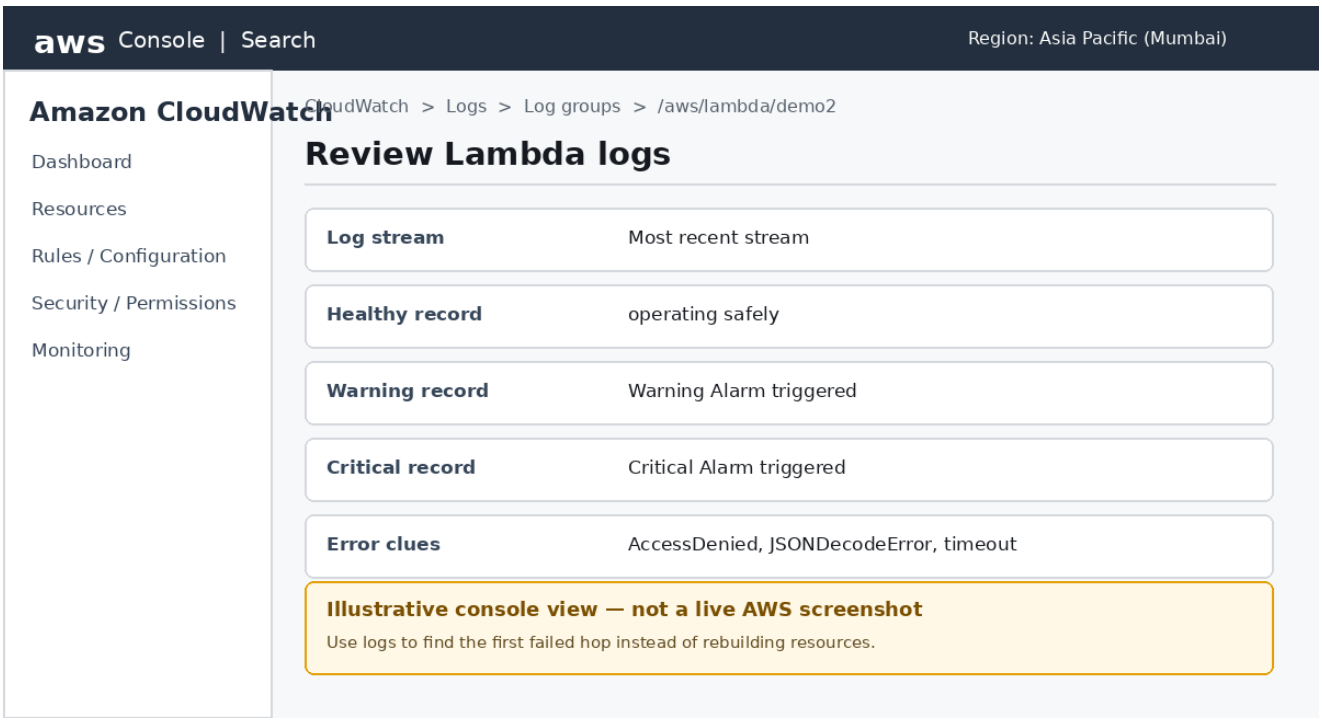
Order	Action	Success evidence
1	Start MQTT Test Client and subscribe to sdk/test/python.	JSON messages appear in the console.
2	Run smart_grid_iot_simulator.py.	Terminal reports 15 records published.
3	Check SQS queue metrics / receive messages.	Messages arrive in the selected queue.
4	Open Lambda demo2 → Monitor → View CloudWatch logs.	Logs show telemetry received and status decisions.
5	Check SNS subscription confirmation.	Subscription status is Confirmed.
6	Inspect inbox / spam folder.	Warning and Critical records generate emails.

Test examples

Payload field values	Expected Lambda behavior
Grid_Status=Normal, temp=58.2, current=420	Log safe operation; no SNS alert.
Grid_Status=Warning, temp=66.8, current=510	Publish warning alert.
Grid_Status=Critical, temp=85, current=610	Publish critical alert.
Grid_Status=Normal, temp=80, current=400	Publish critical alert because temp ≥ 80.

The dataset contains a mix of Normal, Warning and Critical records. Actual alert counts depend on the numerical thresholds as well as Grid_Status; status text alone is not the only trigger.

11. Monitor Lambda execution and troubleshoot



Visual guide: CloudWatch Logs: inspect the latest Lambda log stream. (schematic, not a live AWS screenshot)

Troubleshooting checklist

Symptom	Likely checks / fix
Simulator: ModuleNotFoundError: awsiot	Activate the correct venv; install awsiotsdk inside it; run with that venv's Python.
Excel file not found	Check working directory and DATASET_PATH; ensure the .xlsx file is present.
Certificate / key file missing	Check exact filenames and paths; confirm all three TLS files are in place.
AWS_ERROR_MQTT_UNEXPECTED_HANGUP	Verify certificate Active, policy attached, client ID basicPubSub allowed, topic sdk/test/python allowed, region/endpoint match.
No MQTT messages	Subscribe before run; verify endpoint, topic, certificate and policy.
MQTT messages but SQS empty	Verify IoT rule enabled, SQL topic exactly matches, SQS action selected, IAM role has sqs:SendMessage, queue policy allows it.
SQS messages but Lambda not invoked	Check SQS trigger/event source mapping enabled, same region, execution role permissions, and queue metrics.
Lambda error: JSONDecodeError / missing body	Check SQS message body is JSON and matches expected schema. The code expects record['body'].
Lambda AccessDenied on SNS	Grant sns:Publish to the correct topic ARN in the Lambda execution role; verify region/account/topic ARN.
Lambda runs but no email	Confirm SNS email subscription; check inbox/spam; inspect topic and subscription status.
No warning / critical alerts	Check Grid_Status spelling/case and numeric field names; thresholds are temp ≥65/80 and current ≥500/600.

Useful CloudWatch clues

The supplied handler prints received telemetry, critical/warning alarm messages, or a safe-operation message. If no logs appear at all, Lambda may not be invoked. If logs appear but SNS errors occur, focus on

the topic ARN and execution-role permissions.

12. AWS CLI: optional verification

The workshop pipeline can be configured in the AWS Console; AWS CLI is optional. If you installed it during setup, verify it is working before using commands.

```
aws --version
aws sts get-caller-identity
aws configure list
```

Use the CLI only with an AWS profile and region you intend to use. Do not paste access keys into scripts or screenshots. The simulator itself authenticates to AWS IoT Core with its X.509 device certificate and private key; that is distinct from the AWS CLI's IAM credentials.

Region consistency check

Confirm the IoT endpoint, IoT rule, SQS queue, Lambda function and SNS topic are in the intended region. A resource can exist in another region and still appear missing when the console region selector is different.

IAM permissions summary

Identity / role	Minimum purpose
IoT device certificate policy	Allows MQTT connection for the exact client ID and publish to the exact topic.
IoT rule action role	Allows sqs:SendMessage to the intended SQS queue.
Lambda execution role	Allows CloudWatch Logs; allows sns:Publish to the intended SNS topic; supports SQS trigger processing.
Workshop operator IAM user/role	Allows setup and viewing of IoT Core, IAM, SQS, Lambda, SNS and CloudWatch resources as needed.

13. Workshop facilitator run sheet

Phase	Facilitator action	Participants should see
A. Explain architecture	Walk through Excel → simulator → IoT Core → SQS → Lambda → SNS.	They can name the role of each service.
B. Prepare Python	Install Python packages and verify imports.	Dependencies OK.
C. Verify IoT	Check Thing/certificate/policy/endpoint; subscribe to MQTT topic.	Live JSON telemetry appears.
D. Route messages	Check IoT rule and SQS action permissions.	SQS message count increases.
E. Process events	Check Lambda SQS trigger and deploy code with SNS ARN configured.	Lambda logs show Normal / Warning / Critical decisions.
F. Deliver alerts	Confirm SNS email subscription and run full dataset.	Warning and Critical emails arrive.
G. Debug	Stop at first broken checkpoint and inspect logs / permissions.	Participants diagnose the failing hop.

Before the workshop

☐ Confirm the AWS region and resource names.
☐ Confirm the device certificate is Active and policy attached.
☐ Confirm the IoT rule is enabled and has a valid SQS action role.
☐ Confirm the SQS trigger is enabled on Lambda demo2.
☐ Replace the SNS ARN placeholder and verify sns:Publish permission.
☐ Confirm SNS email subscription.
☐ Test with the MQTT Test Client open first.
☐ Keep private keys and account-specific identifiers out of shared materials.

After the workshop

Consider disabling or deleting temporary resources to avoid ongoing charges. Before deleting anything, verify that it is not shared with another project. Never delete resources in an unfamiliar account or region.

Appendix A. Lambda code reference and setup notes

The uploaded Lambda file uses `json` and `boto3`. It iterates over `event['Records']`, parses each `record['body']` as JSON, then reads Grid_ID, Transformer_Temp_C, Current_A and Grid_Status. The SNS topic ARN is currently a placeholder and must be configured before alerts can publish.

Recommended small code improvement: use an environment variable for the SNS ARN

```
import os
import json
import boto3

sns_client = boto3.client("sns")
SNS_TOPIC_ARN = os.environ["SNS_TOPIC_ARN"]

def lambda_handler(event, context):
    for record in event.get("Records", []):
        payload = json.loads(record["body"])
        # Keep the existing warning / critical conditions below.
```

This snippet illustrates only the ARN configuration and safe iteration. If you adopt it, retain the complete warning/critical logic from your supplied lambda-code.py; do not replace the whole handler with this shortened example.

SQS event structure expected by this handler

```
{
  "Records": [
    {
      "body": "{\"Grid_ID\": \"GRID_003\", \"Transformer_Temp_C\": 66.8, \"Current_A\": 510, \"Grid_Status\": \"Warning\"}"
    }
  ]
}
```

Note that the outer SQS event is JSON, and its body field is a string containing the telemetry JSON. Lambda's SQS integration builds this event shape automatically; you generally should not manually invoke the function with only the telemetry object when testing this handler.

Appendix B. Quick reference

Item	Value used in supplied guide / workflow
AWS region	ap-south-1 (Asia Pacific — Mumbai)
IoT client ID	basicPubSub
MQTT topic	sdk/test/python
MQTT TLS port	8883
Dataset	smart_grid_dataset_15.xlsx
Dataset records	15
IoT rule SQL	SELECT * FROM 'sdk/test/python'
SQS → Lambda	Lambda event source mapping / SQS trigger
Lambda function	demo2
SNS topic	demo2 in the described workflow; confirm its ARN in your account
Critical threshold	Grid_Status Critical OR temp ≥ 80°C OR current ≥ 600 A
Warning threshold	Grid_Status Warning OR temp ≥ 65°C OR current ≥ 500 A
Python packages	awsiotsdk, pandas, openpyxl
Lambda imports	json, boto3

Appendix C. Official AWS documentation starting points

Use current AWS documentation when console labels or supported runtime versions differ:
• AWS IoT Core: docs.aws.amazon.com/iot
• AWS IoT Rules: docs.aws.amazon.com/iot/latest/developerguide/iot-rules.html
• Amazon SQS: docs.aws.amazon.com/AWSSimpleQueueService
• Lambda with SQS: docs.aws.amazon.com/lambda/latest/dg/with-sqs.html
• Amazon SNS: docs.aws.amazon.com/sns/latest/dg/welcome.html

End of handbook • Keep your certificates and IAM credentials private.